

Modeling Computation

R Pruim

2020-04-15

Contents

Preface	2
1 Alphabets, Languages, and Grammars	2
1.1 Terminology	2
1.2 (Phrase-Structure) Grammars	2
2 Types of Grammars	2
2.1 Backus-Naur Form (BNF) for Context Free Grammars	2
3 Finite State Automata	3
3.1 Graph Representation	3
3.2 Extended transition function	3
3.3 Language Recognition	4
3.4 Equivalent Automata and Grammars	5
3.5 Nondeterministic Automata	5
4 Regular Expressions and Languages	7
4.1 Operations on Languages	7
4.2 Regular Expressions	7
4.3 Recognizing Regular Languages with NFAs	7
4.4 Solution to problem 4.3.9	7
5 Regular Grammars and Automata	7
5.1 Grammars to Automata	7
5.2 Automata to Grammars	7
5.3 Some Review	7
6 Regular Languages	7
6.1 Equivalent Definitions	7
6.2 A language that is not regular	7
6.3 Solutions	7
7 DFA/NFA to Regular Expression	7
7.1 GNFA (Generalized NFA)	7
7.2 Algorithm for converting DFA/NFA to Regular Expression	7
7.3 Examples	7
7.4 Review problems	7
8 Regular Expressions in Python	7
8.1 Getting your python environment working	8

9	Turing Machines	8
9.1	Definition of a 1-tape Turing Machine (Syntax)	8
9.2	Execution of a Turing Machine program (Semantics)	8
9.3	Examples	8
9.4	What Can Turing Machines Do?	8
9.5	Running time of a Turing machine	8

Preface

Placeholder

1 Alphabets, Languages, and Grammars

Placeholder

1.1 Terminology

Exercises

1.2 (Phrase-Structure) Grammars

Examples

1.2.1 Derivations and Languages

1.2.1.1 Exercises

2 Types of Grammars

Placeholder

2.1 Backus-Naur Form (BNF) for Context Free Grammars

BNF for ALGOL 60 identifier

BNF for signed integer

BNF specification for Java

Exercises

3 Finite State Automata

A **finite state automaton** consists of the following:

- a finite set of **states**: S
- a finite **input alphabet**: I
- a **start state**: $s_0 \in S$.
- a set of **accepting states**: $F \subseteq S$
- a **transition function**: $f : S \times I \rightarrow S$

Example (M_0) states: $\{A, B, C, D\}$; input alphabet: $\{0, 1\}$; start state: A ; accepting states: $\{A, D\}$; transition function described in table below

M_0								
state	A	A	B	B	C	C	D	D
letter	0	1	0	1	0	1	0	1
f	A	B	A	C	A	A	C	B

3.1 Graph Representation

1. It is often easier to visualize what is going on if we represent an automaton with a graph. Create a labeled, directed graph with the following properties:
 - There is a vertex for each state, labeled with the state. (Draw this as a circle with the state inside the circle.)
 - There is an edge from state s to state t labeled with letter x if and only if $f(s, x) = t$
 - Accepting states are circled a second time. (We could use shape or color or something else to make it clear which states are accepting states, but double circling is easy.)
 - An extra arrow (not coming from any state) points to the start state. This isn't really an edge in the graph, just an extra bit of labeling.

3.2 Extended transition function

We can extend the transition function to $f^* : S \times I^* \rightarrow S$ with the following recursive definition:

- $f^*(s, \lambda) = s$ for any state s
 - $f^*(s, xa) = f(f^*(s, x)a)$ for any $x \in I^*$ and $a \in I$
2. Use this definition to determine the following.
 - a. $f^*(B, \lambda)$
 - b. $f^*(B, 0)$
 - c. $f^*(B, 010)$
 - d. $f^*(A, 101)$
 - e. $f^*(A, 1011)$

3.3 Language Recognition

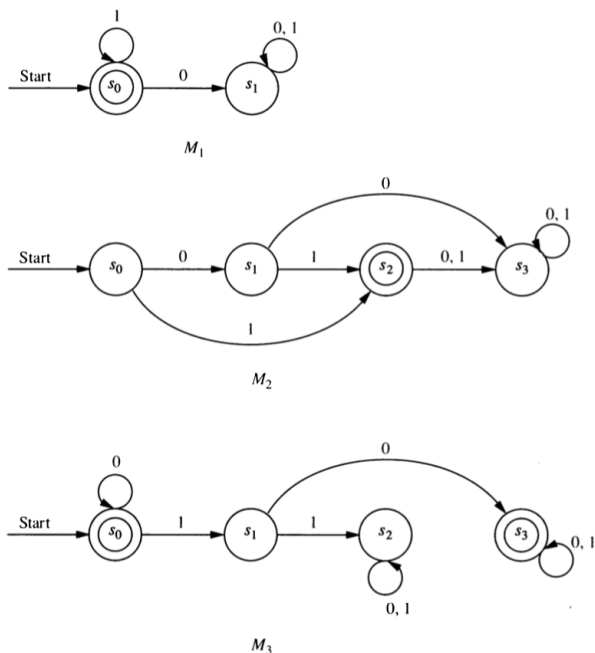
For any automaton M with transition function f , start state s and accepting states F , the language recognized by M (written $L(M)$) is defined as follows:

$$x \in L(M) \iff f^*(s, x) \in F$$

3. For each string x below, compute $f^*(s, x)$. Which of these strings are in $L(M_0)$?
(We will say that such strings are **accepted by** M_0 .)

- a. λ
- b. 0
- c. 1
- d. 010
- e. 1011

Here are three automata:

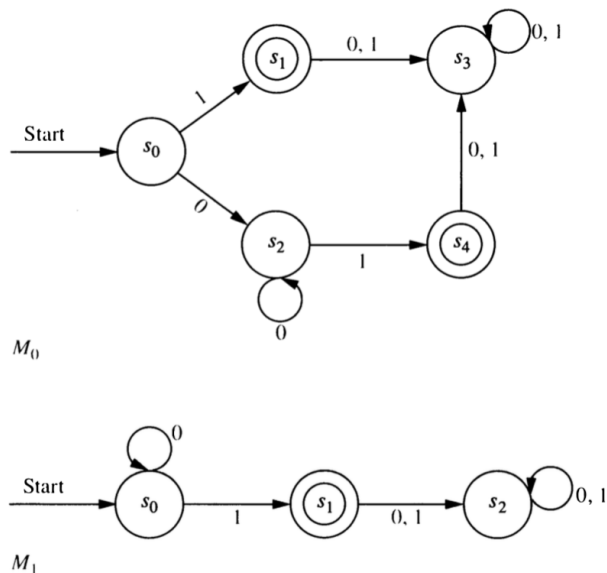


4. For each of the machines M_1, M_2, M_3 , compute
- a. $f^*(s_0, 10)$
 - b. $f^*(s_0, 1011)$
 - c. $f^*(s_1, 1011)$
5. For each of the machines M_1, M_2, M_3 , determine the language recognized.
6. Create automata that recognize each of the following languages.
- a. The set of bitstrings that begin with two 0's
 - b. The set of bitstrings that contain exactly two 0's
 - c. The set of bitstrings that contain at least two 0's
 - d. The set of bitstrings that contain two consecutive 0's (anywhere in the string)
 - e. The set of bitstrings that do not contain two consecutive 0's anywhere
 - f. The set of bitstrings that end with two 0's

3.4 Equivalent Automata and Grammars

We will say that two automata M and N are equivalent if $L(M) = L(N)$, that is, if they recognize the same language. Similarly, two grammars are equivalent if they generate the same language.

7. What must you do to show that two automata are **not** equivalent?
8. What must you do to show that two automata **are** equivalent.
9. Are the two automata represented below equivalent? Explain.



3.5 Nondeterministic Automata

The finite state automata we have seen so far are often called **deterministic finite-state automata** or DFAs. There is a generalization called a **non-deterministic finite-state automaton** or NFA. The only difference is how the transition function is specified. For an NFA, the transition function has the form

$$f : S \times I \rightarrow P(S)$$

where S is the set of states, I is the input alphabet, and $P(S)$ is the powerset of S . (The powerset of S is the set of all subsets of S .) So the transition function for an NFA returns a *set* of states. That set could contain just one state, or multiple states, or no states at all (empty set).

Here is a tabular description of the transition function for an NFA N_0 with start state A and accepting states C and D.

N_0								
state	A	A	B	B	C	C	D	D
letter	0	1	0	1	0	1	0	1
f	$\{A, B\}$	$\{D\}$	$\{A\}$	$\{B, D\}$	$\{\}$	$\{A, C\}$	$\{A, B, C\}$	$\{B\}$

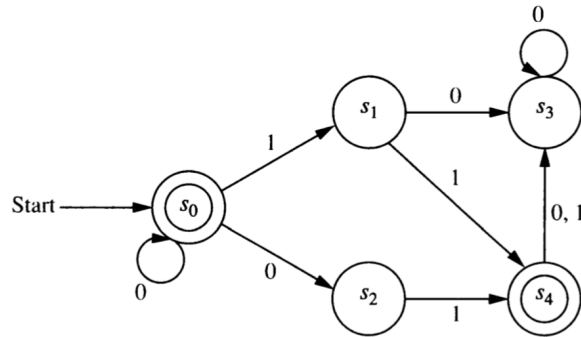
10. Draw the graph representation. How will the graph of an NFA differ from the graph for a DFA?

11. How do we define the language recognized by an NFA? [Check your answer before proceeding.]

12. Which of these strings are accepted by N_0 ?

- a. 01
- b. 011
- c. 00
- d. 0000
- e. λ
- f. 010101

Here is the graph of an NFA N_1 .



13. Make a table showing the transition function for N_1 .

14. Which of the following strings are accepted by N_1 ?

- a. λ
- b. 00
- c. 01
- d. 010
- e. 011
- f. 001

15. Which is harder: to convince someone that a string *is* accepted by an NFA or that it *is not* accepted by an NFA? Why?

16. What is $L(N_1)$?

17. For each of the following languages

- Construct an NFA that recognizes the language.
- Construct a DFA that recognizes the language.

In each case, try to do this with as few states and transitionas as you can. You may find it helpful to give your states good names that reflect what they represent.

- a. The set of bitstrings that either start 01 or start 10.
- b. The set of bitstrings that end with two 0's
- c. The set of bitstrings that either end 01 or end 10.

Which do you find easier to construct: NFAs or DFAs? Why?

18. Create a DFA that recognizes $L(N_1)$.

19. Given an NFA N , is it always possible to create a DFA M that recognizes the same language? If so, explain how. If not, provide an example N and explain why $L(N)$ cannot be recognized by a DFA.

4 Regular Expressions and Languages

Placeholder

4.1 Operations on Languages

4.2 Regular Expressions

4.2.1 Syntax

4.2.2 Semantics

4.3 Recognizing Regular Languages with NFAs

4.4 Solution to problem 4.3.9

5 Regular Grammars and Automata

Placeholder

5.1 Grammars to Automata

5.2 Automata to Grammars

5.3 Some Review

6 Regular Languages

Placeholder

6.1 Equivalent Definitions

Your Mission (You must choose to accept it)

6.2 A language that is not regular

6.3 Solutions

7 DFA/NFA to Regular Expression

Placeholder

7.1 GNFA (Generalized NFA)

7.2 Algorithm for converting DFA/NFA to Regular Expression

7.3 Examples

7.4 Review problems

8 Regular Expressions in Python

Placeholder

8.1 Getting your python environment working

8.1.1 repl.it

8.1.2 Thonny

8.1.3 Testing your python setup

9 Turing Machines

Placeholder

9.1 Definition of a 1-tape Turing Machine (Syntax)

9.2 Execution of a Turing Machine program (Semantics)

9.2.1 ASCII Representations of Turing Machines

9.3 Examples

9.3.1 Example 1

9.3.2 Example 2

9.3.3 The language recognized by a Turing Machine

9.3.4 Designing a Turing Machine

9.4 What Can Turing Machines Do?

9.4.1 Turing Computable Languages

9.4.2 The Halting Problem

9.4.3 Variations on the Turing Machine

9.5 Running time of a Turing machine

9.5.1 P and NP